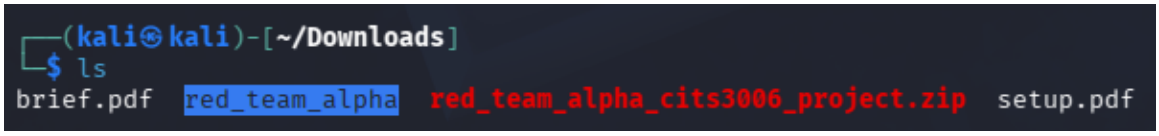
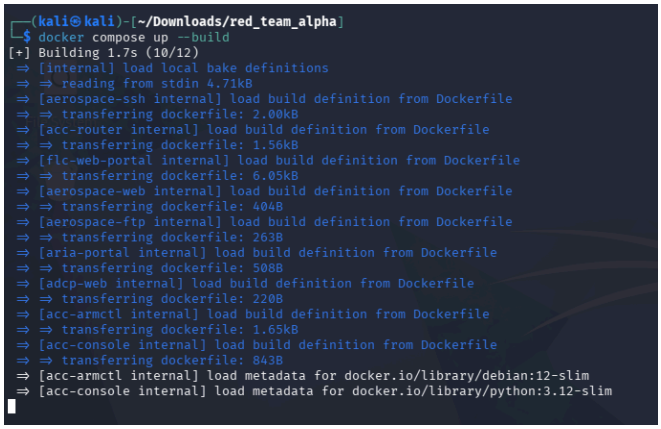
Setup Instructions

All of the required documents should have been provided in a ZIP archive. This archive should include the following files:

- These setup instructions (setup.pdf)
- The mission brief (brief.pdf)
- A directory named red_team_alpha



The red_team_alpha directory is the starting point for the entire vulnerable application. Once you have entered this directory, you need to run `docker compose up --build`.

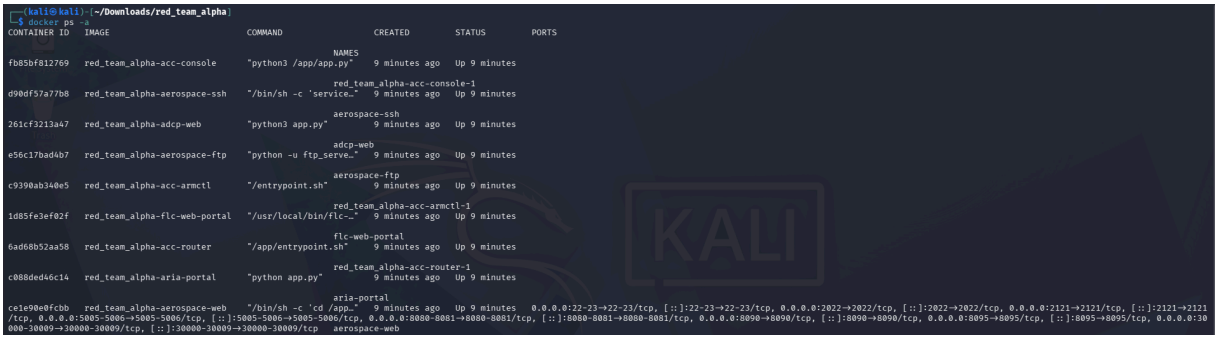


This will begin starting each of the containers required for the application to run, which may take a few minutes depending on the images you have already downloaded (it only requires the python:3.12-slim and debian:12-slim images as the base for each container). Once the containers have all been set up, you should see that several Flask applications have started. You can ignore what the output specifically says, but you must wait until this output is shown before you can begin. It will look something like the following, but not exactly due to slight differences in the order the containers start.

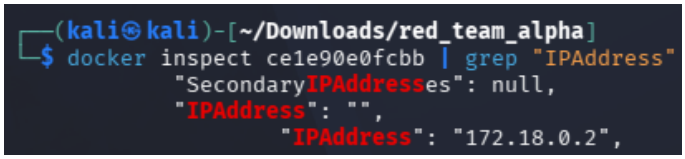


Now that the containers have started, the next thing you need to do is find the IP address of the running containers. Although several containers are created in order to support running this

application, they have been networked together so that they have a single IP address in order to make navigating to each component easier. There are several ways to find the IP address of the containers. The easiest way is to look at the address the web applications are running on and find the container address - in the above screenshot, the container uses the IP 172.18.0.2. The IP address can also be found by running docker inspect <container_ID> from a separate terminal session. The container ID can be found by running docker ps -a and looking for the container with all the open ports.

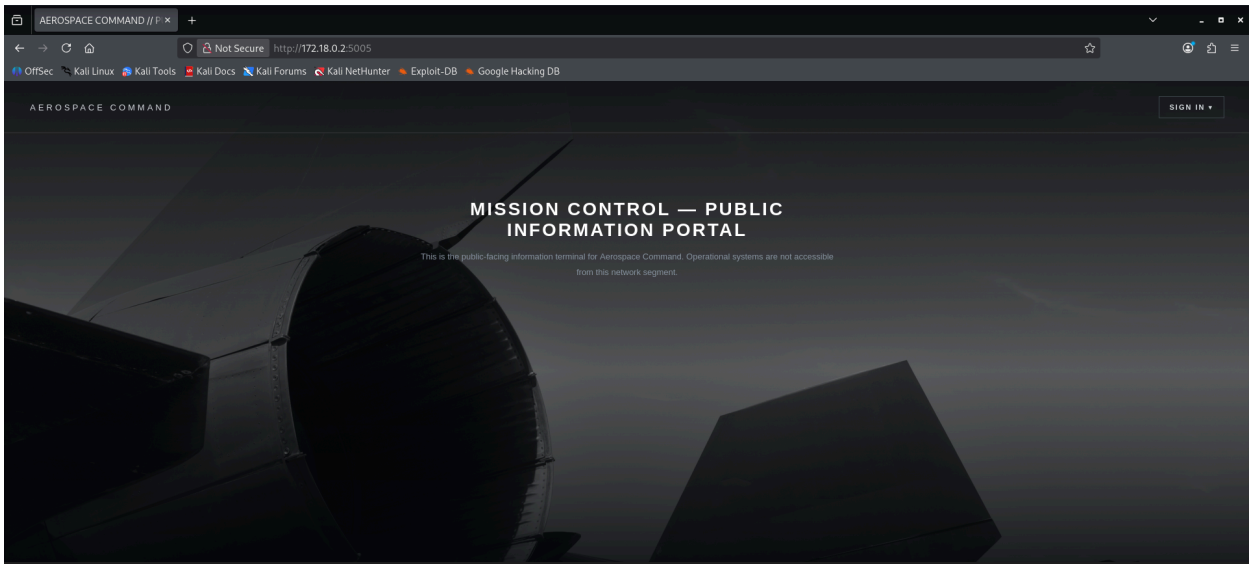


In the above screenshot, the target container has the ID ce1e90e0fcbb. You should now run docker inspect <container_ID> and look for the container's IP address in the output.



In this case, the IP address has been found to be 172.18.0.2 - the exact same IP address identified earlier. Note that while localhost can be used as the IP address, this may potentially introduce noise during certain challenges or your host may be utilising certain ports that this application relies on. Therefore, we recommend that you use the container's IP address when accessing the application.

Now that the IP address has been found, you should navigate to http://IP:5005 on your system's web browser. This will take you to the following web page.



This is the starting point for every vulnerability in the application; the information for every vulnerability can be found from this web application, and any necessary information to complete the challenges will be provided to you.

Troubleshooting

There are a few problems that may occur while trying to set up the application. Although we cannot address every problem you may run into, we have provided solutions for some potentially common issues that may be helpful to you. If you have already got the application running, you can skip this section.

This application requires both Docker and the Docker Compose plugin. If you do not have these packages installed, you should follow the documentation provided by Docker for this purpose (https://docs.docker.com/engine/install/) which explains how to install the necessary Docker packages on your system.

While starting the container, you may run into port conflicts if those ports are already being occupied by your system. Our application requires several ports, which includes 5005, 22, 23, 2022, 2121, 5006, 8080, 8081, 8090, 8095, and the range in 30000-30009. If the containers cannot be started due to a port conflict, you should either stop the applications on your system occupying these ports or modify the `docker-compose.yml` file to change the exposed ports. The required ports are listed under the `aerospace-web` service, and are written in the format `host_port:container_port` (e.g. `80:81` will map port 81 from the container to port 80 on the host). You should only modify the `host_port` portion, as modifying the `container_port` will prevent the application from working.

This application is only confirmed to work on x86-64 based systems. Although it is likely to work on ARM-based systems, it also may not. Nevertheless, the reverse engineering challenges all require analysing binaries compiled using `gcc` on x86-64 systems, so ARM-based users will likely have trouble running and inspecting these. Note that while these binaries have been compiled for x86-64 systems, they do not all require reading assembly and any assembly instructions required to understand also exist in x86 assembly.

If the container isn't behaving as expected and the building process doesn't work, you should ensure that nothing is being cached by your system. The container can be rebuilt ensuring it doesn't use anything in the cache by running `docker compose down`, `docker compose build --no-cache`, and `docker compose up` in that order. This will cause the Docker daemon to rebuild the containers without using its cache, which may allow it to build properly.